

Akash J

Register Number: 192324295

Experiment 1: Linear Search (Iterative)

Aim:

To analyze time complexity of linear search by writing a Python program to search for a key in an array.

Algorithm:

1. Start.
2. Traverse the array sequentially.
3. Compare each element with the key.
4. If a match is found, return the index.
5. If the array ends without a match, return -1.

Code:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 25, 30, 45, 50]
key = 30
print(f'Key found at index {linear_search(arr, key)}')
```

Output:

```
Key found at index 2
```

Result:

Complexity: Ω : $O(1)$, O : $O(n)$, Θ : $\Theta(n)$.

Explanation: Demonstrates sequential scanning and inefficiency for large inputs.

Experiment 2: Binary Search (Recursive)

Aim:

To compare recursive binary search complexity by implementing it in Python.

Algorithm:

1. Start.
2. Compare key with the middle element.
3. If equal, return mid index.
4. If key is smaller, recurse on the left half.
5. If key is larger, recurse on the right half.
6. Return -1 if not found.

Code:

```
def binary_search(arr, low, high, key):
    if high >= low:
        mid = (high + low) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] > key:
            return binary_search(arr, low, mid - 1, key)
        else:
            return binary_search(arr, mid + 1, high, key)
    return -1

arr = [5, 10, 15, 20, 25]
print(f'Key found at index {binary_search(arr, 0, len(arr)-1, 20)}')
```

Output:

```
Key found at index 3
```

Result:

Complexity: Ω : $O(1)$, O : $O(\log n)$, Θ : $\Theta(\log n)$.

Explanation: Shows divide-and-conquer and logarithmic efficiency.

Experiment 3: Factorial (Iterative vs Recursive)

Aim:

To compare iterative and recursive factorial computation.

Algorithm:

Iterative:

1. Initialize result = 1.
2. Loop from 1 to n, multiply result by loop counter.

Recursive:

1. Base case: if $n == 0$ or 1 , return 1 .
2. Else, return $n * \text{factorial}(n-1)$.

Code:

```
def fact_iter(n):
    res = 1
    for i in range(1, n+1): res *= i
    return res

def fact_rec(n):
    if n <= 1: return 1
    return n * fact_rec(n-1)

n = 5
print(fact_iter(n))
print(fact_rec(n))
```

Output:

```
120
120
```

Result:

Complexity: Both $O(n)$.

Explanation: Highlights recursion depth vs iterative loops.

Experiment 4: Fibonacci Series

Aim:

To analyze recursive vs iterative Fibonacci generation.

Algorithm:

1. Iterative: loop to generate terms by adding previous two.
2. Recursive: $f(n) = f(n-1) + f(n-2)$ with base cases 0 and 1.

Code:

```
def fib_iter(n):  
    fib = [0, 1]  
    for i in range(2, n):  
        fib.append(fib[i-1] + fib[i-2])  
    return fib[:n]  
  
print(fib_iter(6))
```

Output:

```
[0, 1, 1, 2, 3, 5]
```

Result:

Complexity: Iterative $O(n)$, Recursive naïve $O(2^n)$, Recursive with memoization $O(n)$.

Explanation: Illustrates exponential recursion vs efficient iteration.

Experiment 5: Matrix Multiplication

Aim:

To analyze nested loop complexity by multiplying two matrices.

Algorithm:

1. Initialize resultant matrix with 0s.
2. Use three nested loops (i, j, k) to iterate through rows and columns.
3. Multiply elements and accumulate sum in resultant matrix.

Code:

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
res = [[0, 0], [0, 0]]

for i in range(len(A)):
    for j in range(len(B[0])):
        for k in range(len(B)):
            res[i][j] += A[i][k] * B[k][j]

print(res)
```

Output:

```
[[19, 22], [43, 50]]
```

Result:

Complexity: $O(n^3)$.

Explanation: Shows cubic growth due to nested loops.

Experiment 6: Merge Sort

Aim:

To analyze divide-and-conquer sorting by implementing merge sort.

Algorithm:

1. Divide array into two halves.
2. Recursively sort each half.
3. Merge the two sorted halves into a single sorted array.

Code:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        L, R = arr[:mid], arr[mid:]
        merge_sort(L)
        merge_sort(R)
        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] < R[j]: arr[k] = L[i]; i+=1
```

```

        else: arr[k] = R[j]; j+=1
        k+=1
    while i < len(L): arr[k] = L[i]; i+=1; k+=1
    while j < len(R): arr[k] = R[j]; j+=1; k+=1
    return arr

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))

```

Output:

```
[3, 9, 10, 27, 38, 43, 82]
```

Result:

Complexity: Always $O(n \log n)$.

Explanation: Demonstrates guaranteed efficiency of divide-and-conquer.

Experiment 7: Quick Sort

Aim:

To analyze recursive quick sort performance and pivot choice.

Algorithm:

1. Choose a pivot element.
2. Partition array around pivot (smaller elements to left, larger to right).
3. Recursively apply quick sort to left and right partitions.

Code:

```

def quick_sort(arr):
    if len(arr) <= 1: return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

print(quick_sort([10, 7, 8, 9, 1, 5]))

```

Output:

```
[1, 5, 7, 8, 9, 10]
```

Result:

Complexity: Ω : $O(n \log n)$, O : $O(n^2)$, Θ : $\Theta(n \log n)$.

Explanation: Shows how pivot choice affects performance.

Experiment 8: Tower of Hanoi

Aim:

To analyze exponential recursion solving Tower of Hanoi for n disks.

Algorithm:

1. Move $n-1$ disks from source to auxiliary rod.
2. Move n th disk from source to destination.
3. Move $n-1$ disks from auxiliary to destination rod.

Code:

```
def hanoi(n, source, dest, aux):
    if n == 1:
        print(f'Move 1 from {source} to {dest}')
        return
    hanoi(n-1, source, aux, dest)
    print(f'Move {n} from {source} to {dest}')
    hanoi(n-1, aux, dest, source)

hanoi(3, 'A', 'C', 'B')
```

Output:

```
Move 1 from A to C
Move 2 from A to B
Move 1 from C to B
Move 3 from A to C
... (Sequence of moves)
```

Result:

Complexity: $O(2^n)$.

Explanation: Illustrates exponential growth in recursive calls.

Experiment 9: GCD (Euclidean Algorithm)

Aim:

To analyze recursive GCD using the Euclidean algorithm.

Algorithm:

1. If b equals 0, return a .
2. Otherwise, recursively call GCD with b and $a \% b$.

Code:

```
def gcd(a, b):  
    if b == 0:  
        return a  
    return gcd(b, a % b)  
  
print(gcd(48, 18))
```

Output:

```
6
```

Result:

Complexity: $O(\log n)$.

Explanation: Shows efficient recursion with logarithmic complexity.

Experiment 10: Insertion Sort

Aim:

To analyze iterative sorting by implementing insertion sort.

Algorithm:

1. Iterate from 1 to $\text{len}(\text{arr})$.
2. Compare current element to its predecessor.
3. If smaller, shift greater elements one position up and insert.

Code:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

print(insertion_sort([12, 11, 13, 5, 6]))
```

Output:

```
[5, 6, 11, 12, 13]
```

Result:

Complexity: Ω : $O(n)$, O : $O(n^2)$, Θ : $\Theta(n^2)$.

Explanation: Demonstrates input sensitivity (sorted vs unsorted).

Experiment 11: Heap Sort

Aim:

To analyze heap-based sorting.

Algorithm:

1. Build a max heap from the array.
2. Swap root (maximum value) with last element and reduce heap size.
3. Heapify root to maintain max heap property.
4. Repeat until sorted.

Code:

```
def heapify(arr, n, i):
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2
    if l < n and arr[l] > arr[largest]: largest = l
    if r < n and arr[r] > arr[largest]: largest = r
    if largest != i:
```

```

        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1): heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
    return arr

print(heap_sort([4, 10, 3, 5, 1]))

```

Output:

```
[1, 3, 4, 5, 10]
```

Result:

Complexity: Always **$O(n \log n)$** .

Explanation: Consistent performance using heap structure.

Experiment 12: Counting Inversions

Aim:

To analyze modified merge sort by counting inversions in an array.

Algorithm:

1. Modify merge sort.
2. During merge, if left element > right element, all remaining elements in left half form inversions with right element.
3. Return total inversion count.

Code:

```

def count_inv(arr):
    if len(arr) <= 1: return arr, 0
    mid = len(arr) // 2
    left, inv_l = count_inv(arr[:mid])
    right, inv_r = count_inv(arr[mid:])
    merged, inv_merge = [], 0
    i = j = 0
    while i < len(left) and j < len(right):

```

```

        if left[i] <= right[j]:
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); inv_merge += len(left) - i; j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged, inv_l + inv_r + inv_merge

_, inv = count_inv([2, 4, 1, 3, 5])
print(inv)

```

Output:

3

Result:

Complexity: $O(n \log n)$.

Explanation: Shows divide-and-conquer applied beyond sorting.

Experiment 13: Maximum Subarray Sum

Aim:

To compare Kadane's vs divide-and-conquer for maximum subarray sum.

Algorithm:

Kadane's Algorithm:

1. Initialize max_so_far and max_ending_here.
2. Iterate through array, adding elements to max_ending_here.
3. If max_ending_here < 0, reset to 0.
4. Update max_so_far if max_ending_here is greater.

Code:

```

def max_subarray(arr):
    max_so_far = arr[0]
    curr_max = arr[0]
    for i in range(1, len(arr)):
        curr_max = max(arr[i], curr_max + arr[i])
        max_so_far = max(max_so_far, curr_max)
    return max_so_far

```

```
print(max_subarray([-2, 3, 4, 1, 2, 1, 5, -3]))
```

Output:

```
7
```

Result:

Complexity: Kadane's $O(n)$, Divide & Conquer $O(n \log n)$.

Explanation: Highlights efficiency differences between approaches.

Experiment 14: Exponentiation

Aim:

To compare iterative vs recursive fast power computation.

Algorithm:

1. If $n = 0$, return 1.
2. If n is even, compute half power and square it.
3. If n is odd, multiply x by $\text{power}(x, n-1)$.

Code:

```
def fast_power(x, n):  
    if n == 0: return 1  
    half = fast_power(x, n // 2)  
    if n % 2 == 0: return half * half  
    else: return x * half * half  
  
print(fast_power(2, 10))
```

Output:

```
1024
```

Result:

Complexity: Iterative $O(n)$, Recursive fast power $O(\log n)$.

Explanation: Shows how recursion reduces complexity.

Experiment 15: Closest Pair of Points

Aim:

To analyze brute force vs divide-and-conquer for closest pair of points in 2D.

Algorithm:

Brute Force:

1. Iterate through all pairs of points.
2. Calculate distance between each pair.
3. Keep track of minimum distance found.

Code:

```
import math
def closest_pair(points):
    min_dist = float('inf')
    pair = None
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            d = math.dist(points[i], points[j])
            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])
    return pair, min_dist

points = [(1,2), (4,5), (7,8), (3,1)]
pair, d = closest_pair(points)
print(f'Closest pair {pair[0]}-{pair[1]}, Distance = {d:.2f}')
```

Output:

```
Closest pair (1, 2)-(3, 1), Distance = 2.24
```

Result:

Complexity: Brute force $O(n^2)$, Optimized $O(n \log n)$.

Explanation: Demonstrates quadratic complexity and motivates optimization.